Un dato, varios lenguajes

Seguimos a Ana: de un registro en memoria a una tarjeta web que responde a un clic.

01 · REPRESENTAR

JSON y XML describen información. Un esquema establece su estructura y sus tipos.

02 · TRANSPORTAR

Serializar convierte los datos en bytes que otra aplicación puede recibir y reconstruir.

03 · PRESENTAR

HTML aporta estructura y significado. CSS declara el aspecto y la distribución.

04 · ACTUAR

El navegador crea el DOM. JavaScript consulta sus objetos y modifica la página.

Ana · 20 años → **Mensaje** → **Tarjeta web**

No todo lenguaje es un lenguaje de programación. Podemos representar datos, marcar documentos o declarar estilos sin escribir un algoritmo.

Pregunta guía: ¿estoy describiendo información, declarando un resultado o programando un comportamiento?

XML y JSON: el mismo dato

Dos formas textuales de representar información estructurada.

JSON · claves y valores

```
{
  "nombre": "Ana",
  "edad": 20
}
```

XML · elementos y atributos

```
<alumno>
  <nombre>Ana</nombre>
  <edad>20</edad>
</alumno>
```

JSON

Objetos, listas, cadenas, números, booleanos y null. Habitual en APIs y configuraciones. No es código JavaScript ejecutable.

XML

Etiquetas extensibles, atributos y texto. Útil en documentos, integraciones y formatos como SVG. XSD puede aportar tipos.

¿Datos, programa o base de datos? Depende del uso. XSLT expresa transformaciones con sintaxis XML. Guardar registros en XML no aporta por sí solo un gestor con consultas, índices o transacciones.

La sintaxis no decide todo: el significado lo define el formato concreto y el sistema que lo interpreta.

Las etiquetas también viajan

Los nombres de campo ayudan a entender el mensaje, pero cuestan bytes.

```
{"nombre":"Ana","edad":20}
```

26 bytes

JSON compacto en UTF-8: sin espacios adicionales ni salto final.

5 bytes	21 bytes
Ana + 20 Caracteres de los valores.	Nombres + sintaxis Claves, comillas, llaves, comas y dos puntos.

¿Un problema real?

En millones de mensajes, el tamaño y el trabajo de codificar y decodificar pueden importar. Pero la legibilidad facilita depurar, y gzip o Brotli aprovechan las repeticiones. La mejor opción se mide con datos reales.

Ejemplo ilustrativo, no benchmark: estos 26 bytes no incluyen HTTP, TLS, cabeceras ni otros costes de red.

El esquema es un contrato

Antes de hablar, acordamos qué campos existen y qué valores admiten.



Contrato de alumno

nombre: texto obligatorio · edad: entero obligatorio, mínimo 0.
Un dato puede cumplir estas reglas y aun así ser falso.

```
{
  "type": "object",
  "properties": {
    "nombre": {"type": "string"},
    "edad": {"type": "integer", "minimum": 0}
  },
  "required": ["nombre", "edad"]
}
```

Arriba: JSON Schema. En XML, XSD permite validar estructura y tipos; DTD define estructura con menos riqueza de tipos.

Atención: añadir JSON Schema no reduce el JSON transmitido. Para cambiar los bytes hay que cambiar la codificación.

Binario no significa comprimido

Conocer los tipos permite una codificación compacta; no implica tamaños fijos.

```
syntax = "proto3";
message Alumno {
  string nombre = 1;
  uint32 edad = 2;
}
```

Protobuf · Ana, 20 → 7 bytes

0A	03	41 6E 61	10	14
campo 1	longitud	Ana	campo 2	20

CODIFICAR

Las claves textuales se sustituyen por etiquetas numéricas. Las cadenas llevan longitud; muchos enteros usan varints: tamaño variable.

COMPRIMIR

gzip, Brotli o Zstandard explotan patrones. Pueden aplicarse a texto o binario. En mensajes pequeños, su sobrecarga puede no compensar.

Comparación de este único mensaje: 26 bytes de JSON frente a 7 de Protobuf, sin esquema, envoltorio ni transporte.

No son casillas siempre iguales: el esquema guía la interpretación; la longitud también puede viajar en el mensaje.

Fuente: protobuf.dev/programming-guides/encoding/

Elegir cómo serializar

Los microservicios pueden intercambiar JSON o usar contratos y formatos binarios.

JSON / XML

Texto legible; amplias herramientas. Esquemas opcionales para validar.

Protocol Buffers

Contrato .proto y campos numerados. Frecuente con gRPC.

Avro

Esquema normalmente en JSON. En binario omite nombres de campo; necesita el esquema del escritor para leer.

MessagePack / CBOR

Formatos binarios que no exigen un esquema externo. Los mapas pueden seguir incluyendo claves textuales.

BSON / FlatBuffers

BSON incorpora tipos y longitudes, usado en MongoDB. FlatBuffers permite acceder a datos sin reconstruir todo el objeto.

Elegir exige medir: tamaño, latencia, CPU, facilidad de depuración y evolución del contrato. Binario no garantiza ser siempre más pequeño ni más rápido.

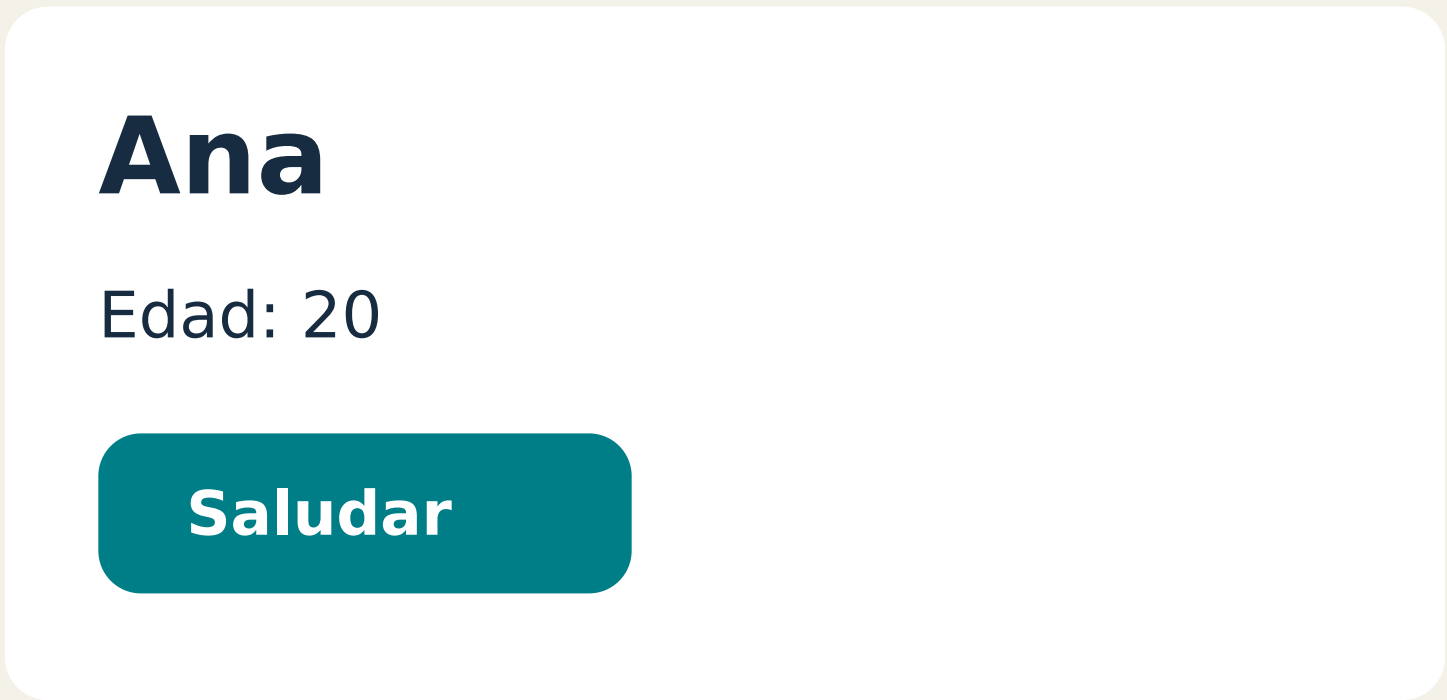
Evolución: emisor y receptor pueden tener versiones distintas. Hay que mantener compatibilidad; en Protobuf no reutilices números de campos eliminados.

Fuentes: avro.apache.org/docs/1.12.0/specification/ · msgpack.org · RFC 8949

HTML: estructura con significado

Se parece a XML por sus etiquetas, pero tiene reglas propias. XHTML sí sigue la sintaxis XML.

```
<article class="tarjeta">
  <h1 id="nombre">Ana</h1>
  <p>Edad: <span>20</span></p>
  <button id="saludar">Saludar</button>
</article>
```



No todo es un div. article, h1, p y button expresan una función. div es un contenedor genérico; span agrupa contenido en línea. Elegir etiquetas adecuadas ayuda a la accesibilidad.

HTML dice qué es cada cosa. CSS controla cómo se presenta. Que un elemento parezca una caja no lo convierte en un div.

Cajas: block e inline

En el flujo normal y con escritura horizontal, así se distribuyen habitualmente.

BLOCK

Inicia una nueva línea. Con ancho automático suele ocupar el ancho disponible del contenedor.

Un párrafo

Otro párrafo

INLINE

Participa en una línea de texto. Puede quedar junto a otros fragmentos y saltar de línea si falta espacio.

Hola soy Ana ¿qué tal?

INLINE-BLOCK

Se coloca en línea como una unidad, pero acepta ancho y alto. En un inline de texto normal, width y height no funcionan como en una caja block.

```
.fila {
  display: flex;
  gap: 16px;
  flex-wrap: wrap;
}
```

Para alinear componentes:
Flexbox trabaja sobre un eje principal; Grid organiza filas y columnas. display cambia el comportamiento, no el significado HTML.

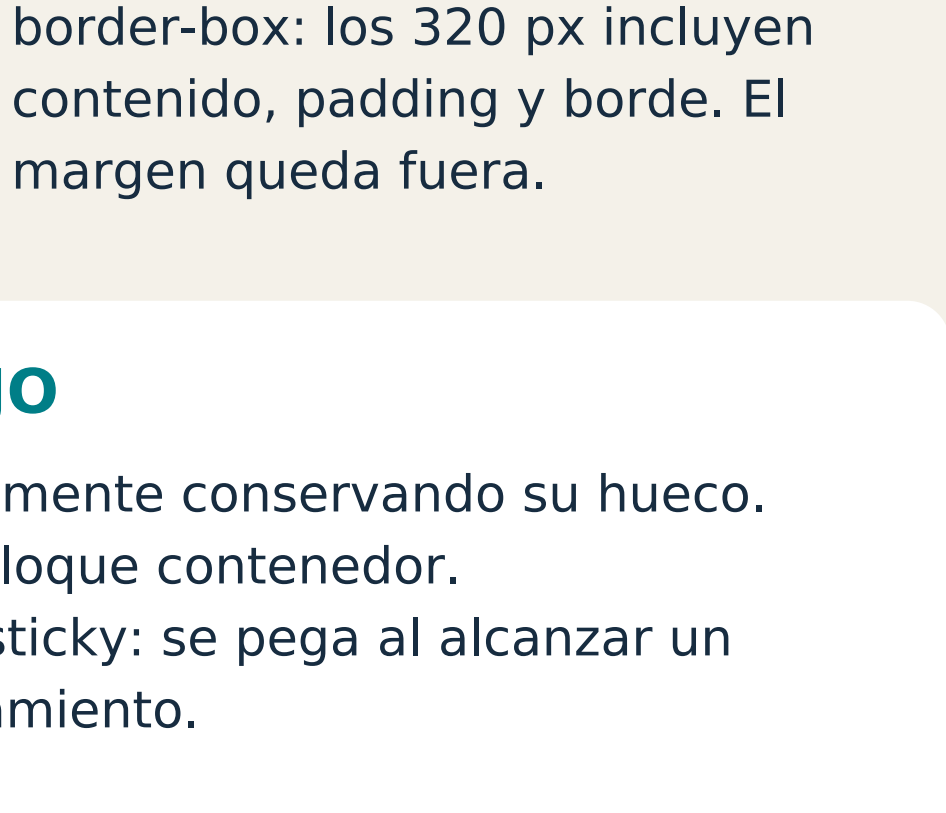
Block no significa “debajo siempre”. Flexbox, Grid y otros contextos de diseño cambian cómo se distribuyen las cajas.

Fuente: MDN · The box model

CSS declara el resultado

No es imperativo: describes reglas y el navegador calcula cómo aplicarlas.

```
.tarjeta {
  color: #172c40;
  background: white;
  padding: 24px;
  border: 2px solid teal;
  margin: 16px;
  width: 320px;
  box-sizing: border-box;
}
```



border-box: los 320 px incluyen contenido, padding y borde. El margen queda fuera.

POSICIONAR SIN PERDER EL FLUJO

static: flujo normal · relative: desplaza visualmente conservando su hueco.
absolute: sale del flujo y se referencia a su bloque contenedor.
fixed: normalmente respecto a la ventana · sticky: se pega al alcanzar un umbral dentro de su contenedor de desplazamiento.

Empieza por Flexbox o Grid para el diseño. Usa position para necesidades concretas; no hace falta colocar todo con coordenadas.

Selectores: a quién afecta CSS

Un id identifica un elemento; una class agrupa elementos reutilizables.

```
<button id="saludar" class="boton destacado">
  Saludar
</button>
```

`button`	Todos los elementos button.
`#saludar`	El elemento con ese id; debe ser único en el documento.
`.boton`	Todos los elementos con esa clase.
`.boton:hover`	Botones con esa clase cuando el puntero está encima.
`.boton:focus-visible`	Estado de foco visible, útil al navegar con teclado.
`.tarjeta > p`	Párrafos que son hijos directos de .tarjeta.
`.tarjeta p`	Párrafos descendientes, aunque haya niveles intermedios.

:hover es una pseudoclase (estado). ::before es un pseudoelemento. No dependas solo del hover: una pantalla táctil puede no ofrecerlo.

Una class se repite y un elemento puede tener varias. Los atributos HTML van sin # ni punto; esos símbolos pertenecen al selector CSS.

Cuando dos reglas compiten

La cascada decide cada propiedad por separado; la especificidad es solo una parte.

Orden general: reglas aplicables → origen, importancia y capas → especificidad → cercanía de @scope → orden de aparición.

Para nuestros ejemplos sencillos

Mismo origen de autor, sin capas, sin !important, sin animaciones, sin transiciones ni @scope:

ID	Clase / :hover	Tipo / ::before
1 · 0 · 0	0 · 1 · 0	0 · 0 · 1

```
button   { color: black; }
.boton   { color: blue;  }
#saludar { color: green; }
```

Gana verde. Se comparan las tres columnas de izquierda a derecha: no son puntos decimales. Si hay empate, gana la declaración posterior.

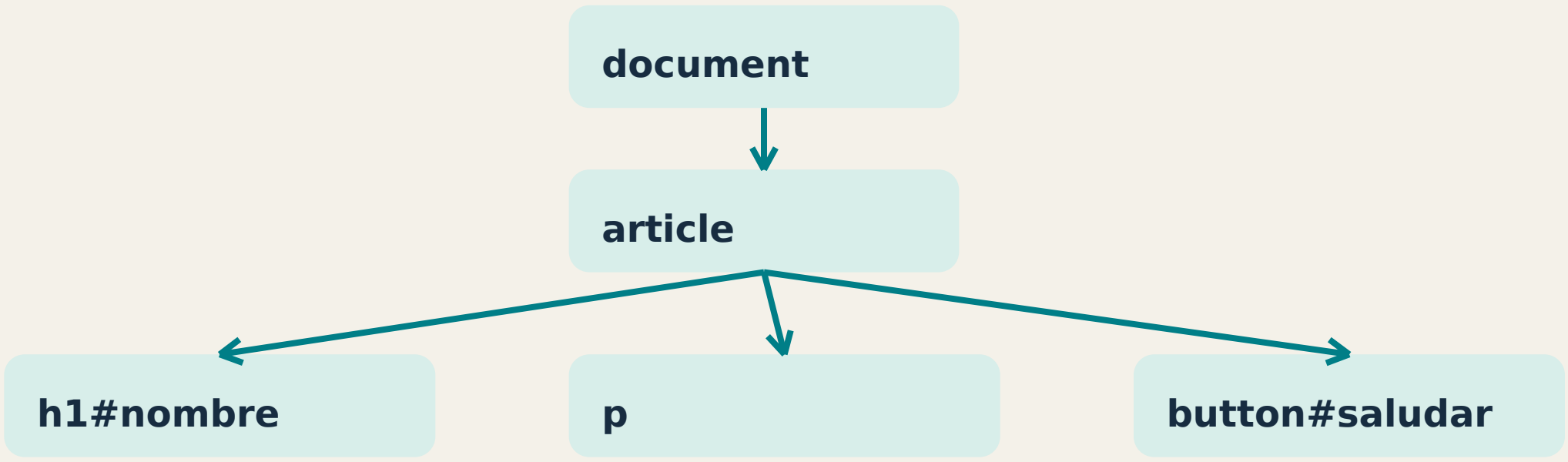
style="..." supera las reglas normales del autor. !important cambia el nivel de prioridad: no suma especificidad ni gana siempre. La herencia aporta valores cuando corresponde; no compite como otro selector.

Para aprender: usa clases y reglas simples. “La última gana” solo es cierto después de resolver los criterios anteriores.

Fuente: developer.mozilla.org · Introduction to the CSS cascade

Del HTML al DOM… y al clic

El DOM es un árbol de objetos en memoria. JavaScript puede consultar y modificar sus nodos.



Árbol simplificado: se omiten html, body, span y nodos de texto. El navegador crea el DOM al analizar el HTML, antes de terminar de pintar.

```
// Ejecutar al final de body, tras el HTML
const nombre = document.getElementById("nombre");
const boton = document.querySelector("#saludar");
boton.addEventListener("click", () => {
  nombre.textContent = "¡Hola, Ana!";
  nombre.classList.toggle("resaltado");
});
```

```
/* CSS: se activa al añadir la clase */
.resaltado { color: teal; }
```

DOM + estilos → diseño → pintura. Un cambio puede activar nuevos cálculos. JavaScript modifica objetos, no los píxeles ni el archivo HTML original.

Fuente: MDN · Document Object Model (DOM). Referencia enlazada en este pie.